

Министерство образования Республики Беларусь  
Учреждение образования  
«Брестский государственный технический университет»  
Кафедра ИИТ

**Лабораторная работа №1**

По дисциплине: «Криптография и сетевая безопасность»

Тема: «Исследование алгоритмов генерации псевдослучайных последовательностей и методов тестирования чисел на простоту»

Выполнил:  
Студент 4-го курса  
Группы АС-66  
Ануфриенко М. А.  
Проверила:  
Хацкевич А. С.

**Цель работы:** освоить программную реализацию и практическое применение алгоритмов генерации псевдослучайных последовательностей (РСП), методов их статистической оценки, а также вероятностных и детерминированных тестов простоты чисел с построением доказуемо простых чисел.

## Вариант 1

### 1. Постановочная часть

#### 1.1. Этап 1. Генерация и тестирование РСП методом Макларена–Марсальи (N=10, K=5)

По таблице варианта 1 требуется реализовать генератор псевдослучайных чисел комбинированием двух простых конгруэнтных генераторов методом Макларена–Марсальи (методичка, §4.8) с параметрами: N=10 — размер алфавита основного генератора G1, K=5 — размер алфавита генератора-селектора G2 и одновременно размер вспомогательной T-таблицы.

Формальная постановка. Пусть G1 — линейный конгруэнтный генератор (ЛКГ) с параметрами  $(x_0, a, c, N)$ , порождающий последовательность  $\{x_i\} \subset A(N) = \{0, \dots, 9\}$  по рекурренте

$$x_{t+1} = (a \cdot x_t + c) \bmod N \quad (\text{формула (1), §4.1})$$

и G2 — ЛКГ с параметрами  $(y_0, a', c', K)$ , порождающий  $\{y_i\} \subset A(K) = \{0, \dots, 4\}$  по тому же правилу. T-таблица из K ячеек инициализируется первыми K значениями  $\{x_i\}$ . Выходная последовательность  $\{z_k\}$  строится правилом (§4.8):

$$s \leftarrow y_k; \quad z_k \leftarrow T(s); \quad T(s) \leftarrow x_{K+k}, \quad k = 0, 1, \dots$$

Требуется:

1. Подобрать параметры  $(a, c, x_0)$  для G1 и  $(a', c', y_0)$  для G2 так, чтобы каждый из подгенераторов достигал максимального периода ( $T_{\max} = N = 10$  и  $T_{\max} = K = 5$  соответственно) согласно достаточному условию §4.1 (свойство 4):  $\text{НОД}(c, N) = 1$ ;  $(a - 1)$  кратно каждому простому делителю N;  $(a - 1)$  кратно 4, если N кратно 4.
2. Провести статистическое тестирование выходной последовательности  $\{z_k\}$ : оценить частотность появления каждого символа алфавита  $\{0, \dots, 9\}$  и проверить гипотезу о равномерности распределения критерием хи-квадрат Пирсона.
3. Продемонстрировать изменение периода и статистических свойств генератора при выполнении и при нарушении условия максимального периода.

#### 1.2. Этап 2. Генерация большого простого числа тестом Соловея–Штрассена

По таблице варианта в ЛР №6 целевым тестом простоты для варианта 1 является тест Соловея–Штрассена (§6.5), применяемый вместо стандартного теста Рабина–Миллера.

Формальная постановка (стандартная схема, Задание ЛР №6, п.1):

1. С помощью генератора Этапа 1 сформировать случайное r-битное число-кандидат n.

2. Установить старший бит  $n$  в 1 (гарантия разрядности  $p$ ) и младший бит в 1 (гарантия нечётности).
3. Отсеять заведомо составные кандидаты пробным делением на простые числа, не превосходящие 2000 (по данным методички это отсекает порядка 80% составных чисел и снижает нагрузку на последующий вероятностный тест).
4. К оставшимся кандидатам применить тест Соловея–Штрассена: для случайного основания  $a \in [2, n-2]$  при  $\text{НОД}(a, n) = 1$  проверяется сравнение

$$a^{((n-1)/2)} \equiv (a/n) \pmod{n} \quad (\text{формула (2), теорема 2, §6.5})$$

где  $(a/n)$  — символ Якоби. Несовпадение означает, что  $n$  составное; совпадение при  $k$  независимых раундах даёт вероятность ошибки не более  $2^{-k}$ . 5. Зафиксировать в логе число отсеянных пробным делением кандидатов, число попыток и время генерации.

Существенное уточнение постановки, выявленное в ходе реализации: генератор Этапа 1 ( $N=10, K=5$ ), хотя и достигает максимального периода по условию §4.1, обладает периодом  $T=10$ , на порядки меньшим, чем  $2^p$  возможных  $p$ -битных чисел уже при  $p \geq 16-20$ . Поэтому для практической генерации кандидатов он используется как источник сива для качественного генератора битов, а не напрямую как побитовый источник — это прямо допускается формулировкой задания («сгенерированные... числа или базовые механизмы генерации»).

## 2. Листинги ключевых фрагментов ПО

Листинг 1. Линейный конгруэнтный генератор и проверка достаточного условия максимального периода (src/rng/lcg.py)

```
class LCG:
    def __init__(self, a: int, c: int, N: int, x0: int):
        if N <= 0:
            raise ValueError("N должно быть положительным")
        self.a = a
        self.c = c
        self.N = N
        self.x0 = x0 % N
        self._x = self.x0
    def next(self) -> int:
        """Сделать один шаг рекурренты и вернуть новое значение x_{t+1}."""
        self._x = (self.a * self._x + self.c) % self.N
        return self._x
    def satisfies_max_period_condition(self) -> bool:
        """
        Достаточное условие максимального периода T_max = N (§4.1, св-во 4):
        а) НОД(c, N) = 1
        б) (a - 1) кратно каждому простому делителю N
        в) (a - 1) кратно 4, если N кратно 4
        """
        from math import gcd
        if gcd(self.c, self.N) != 1:
            return False
        b, n = self.a - 1, self.N
        p = 2
        while p * p <= n:
            if n % p == 0:
```

```

        if b % p != 0:
            return False
        while n % p == 0:
            n //= p
        p += 1
    if n > 1 and b % n != 0:
        return False
    if self.N % 4 == 0 and b % 4 != 0:
        return False
    return True

```

## Листинг 2. Комбинирование методом Макларена–Марсальи (src/rng/mm\_combine.py)

```

class MaclarenMarsaglia:
    def reset(self) -> None:
        """Сбросить оба под-генератора и заново заполнить T-таблицу."""
        self.g1.reset()
        self.g2.reset()
        self._table = [self.g1.next() for _ in range(self.K)]

    def next(self) -> int:
        if self._table is None:
            self.reset()
        s = self.g2.next() % self.K
        z = self._table[s]
        self._table[s] = self.g1.next()
        return z

```

## Листинг 3. Параметры генератора варианта 1 (src/rng/factory.py) — оба под-генератора подобраны по условию §4.1 (свойство 4 вырождается в $a \equiv 1 \pmod{N}$ для $N$ с несколькими простыми делителями)

```

def build_variant1_generator() -> MaclarenMarsaglia:
    g1 = LCG(a=1, c=3, N=10, x0=0) # T_max = N = 10
    g2 = LCG(a=1, c=2, N=5, x0=0) # T_max = K = 5
    assert g1.satisfies_max_period_condition()
    assert g2.satisfies_max_period_condition()
    return MaclarenMarsaglia(g1, g2, K=5)

```

## Листинг 4. Символ Якоби без факторизации $n$ (src/primes/jacobi.py)

```

def jacobi_symbol(a: int, n: int) -> int:
    a = a % n
    result = 1
    while a != 0:
        while a % 2 == 0:
            a //= 2
            if n % 8 in (3, 5):
                result = -result
        a, n = n, a
        if a % 4 == 3 and n % 4 == 3:
            result = -result
        a = a % n
    return result if n == 1 else 0

```

## Листинг 5. Тест Соловея–Штрассена (src/primes/solovay\_strassen.py)

```
def solovay_strassen_round(n: int, rng=None) -> bool:
    """Один раунд теста. True – "вероятно простое", False – точно составное."""
    rng = rng or random
    if n in (2, 3):
        return True
    if n < 2 or n % 2 == 0:
        return False

    a = rng.randrange(2, n - 1)
    if gcd(a, n) != 1:
        return False

    x = jacobi_symbol(a, n)
    if x == 0:
        return False

    return pow(a, (n - 1) // 2, n) == x % n

def is_probable_prime(n: int, rounds: int = 5, rng=None) -> bool:
    """Тест Соловея–Штрассена за k=rounds раундов. Вероятность ошибки <= 2^-rounds."""
    return n >= 2 and all(solovay_strassen_round(n, rng) for _ in range(rounds))
```

## Листинг 6. Генератор Этапа 1 как источник сида для Этапа 2 (src/rng/seeding.py)

```
def seed_from_stage1(gen: MaclarenMarsaglia, digits: int = 64) -> int:
    """Извлечь `digits` цифр из генератора Этапа 1 и собрать из них сид."""
    gen.reset()
    values = gen.sequence(digits)
    return int("".join(str(v) for v in values))

def build_stage2_rng(gen: MaclarenMarsaglia, digits: int = 64) -> random.Random:
    """random.Random, засеянный значением, извлечённым из генератора Этапа 1."""
    return random.Random(seed_from_stage1(gen, digits))
```

## Листинг 7. Полная схема генерации и проверки кандидата (src/primes/candidate.py)

```
def generate_prime(make_candidate, p, ss_rounds=5, rng=None, log=None,
                  max_attempts=1_000_000) -> int:
    """
    1. p-битный кандидат из make_candidate();
    2. просеивание пробным делением на простые < 2000;
    3. финальная проверка тестом Соловея–Штрассена (rounds=ss_rounds).
    """
    attempts = sieved_out = 0
    t0 = time.perf_counter()
    while attempts < max_attempts:
        attempts += 1
        n = make_candidate()
        if not trial_division_ok(n):
            sieved_out += 1
            continue
        if is_probable_prime(n, rounds=ss_rounds, rng=rng):
            if log is not None:
```

```

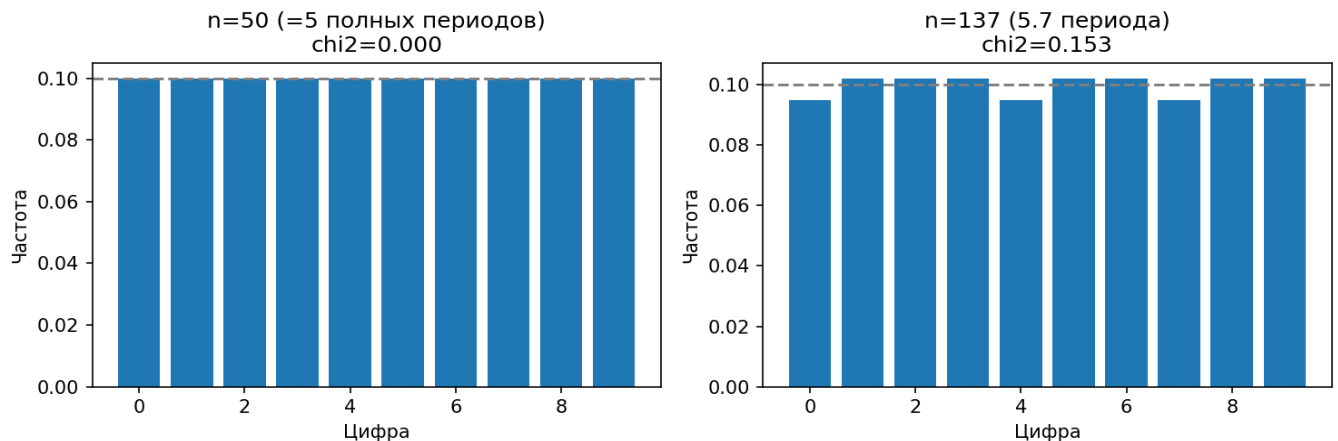
log.append({"n": n, "bits": p, "attempts": attempts,
           "sieved_out": sieved_out,
           "elapsed_sec": time.perf_counter() - t0})

return n
raise RuntimeError(f"Не удалось найти простое число за {max_attempts} попыток")

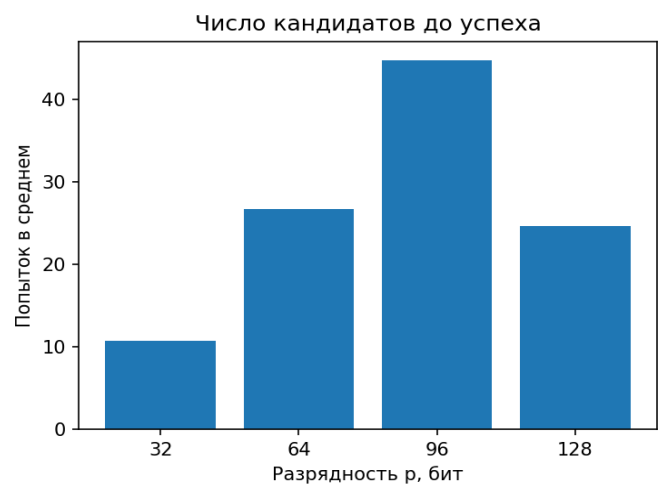
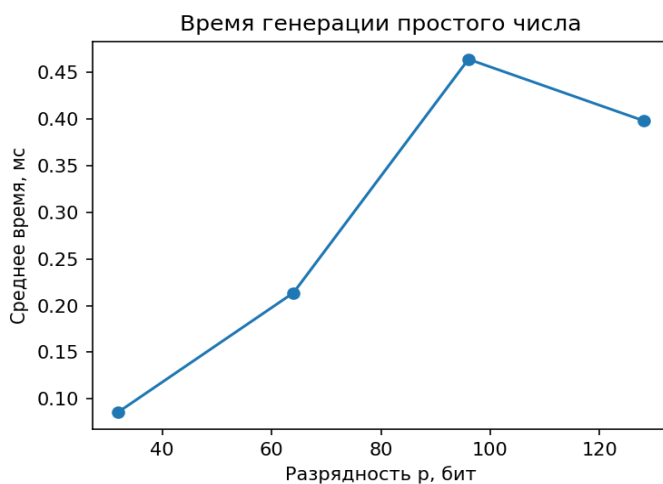
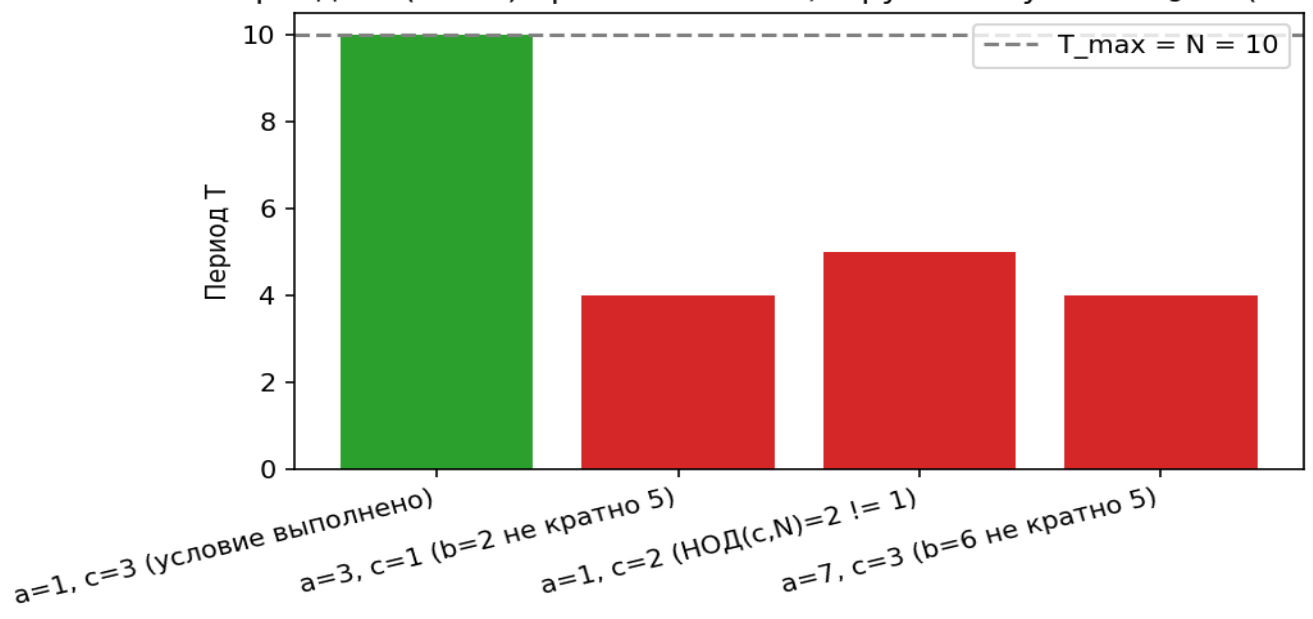
```

### 3. Демонстрация изменения свойств сгенерированных последовательностей при изменении параметров генератора

Частоты выхода ММ-генератора: эффект длины относительно периода  $T=10$



Период G1 ( $N=10$ ) при выполнении/нарушении условия §4.1 (св-во



## Лог работы генератора простых чисел с фиксацией времени и количества отсеянных кандидатов:

```
{
  "frequency_test": {
    "50 (=5 полных периодов)": {
      "freqs": {
        "0": 0.1,
        "1": 0.1,
        "2": 0.1,
        "3": 0.1,
        "4": 0.1,
        "5": 0.1,
        "6": 0.1,
        "7": 0.1,
        "8": 0.1,
        "9": 0.1
      },
      "chi2": 0.0,
      "df": 9,
      "critical_95": 16.919,
      "uniform_hypothesis_accepted": true
    },
    "137 (5.7 периода)": {
      "freqs": {
        "0": 0.0948905109489051,
        "1": 0.10218978102189781,
        "2": 0.10218978102189781,
        "3": 0.10218978102189781,
        "4": 0.0948905109489051,
        "5": 0.10218978102189781,
        "6": 0.10218978102189781,
        "7": 0.0948905109489051,
        "8": 0.10218978102189781,
        "9": 0.10218978102189781
      },
      "chi2": 0.15328467153284672,
      "df": 9,
      "critical_95": 16.919,
      "uniform_hypothesis_accepted": true
    }
  },
  "period_comparison": {
    "a=1, c=3 (условие выполнено)": {
      "period": 10,
      "pre_period": 0,
      "condition_met": true
    },
    "a=3, c=1 (b=2 не кратно 5)": {
      "period": 4,
      "pre_period": 0,
      "condition_met": false
    },
    "a=1, c=2 (НОД(c,N)=2 != 1)": {
      "period": 5,
      "pre_period": 0,
      "condition_met": false
    },
    "a=7, c=3 (b=6 не кратно 5)": {
      "period": 4,
      "pre_period": 0,
      "condition_met": false
    }
  },
  "prime_generation": {
    "summary": {
      "32": {
        "avg_attempts": 10.75,
        "avg_sieved_out": 8.625,
        "avg_time_ms": 0.08592325001188783
      },
      "64": {
        "avg_attempts": 26.75,
        "avg_sieved_out": 23.0,
        "avg_time_ms": 0.21369949999439086
      }
    }
  }
}
```

```

},
"96": {
  "avg_attempts": 44.75,
  "avg_sieved_out": 37.5,
  "avg_time_ms": 0.46421637497928714
},
"128": {
  "avg_attempts": 24.625,
  "avg_sieved_out": 21.125,
  "avg_time_ms": 0.39835012499622735
}
},
"log": [
  {
    "n": 3322233487,
    "bits": 32,
    "attempts": 5,
    "sieved_out": 4,
    "elapsed_sec": 8.653899999444548e-05
  },
  {
    "n": 4134165097,
    "bits": 32,
    "attempts": 1,
    "sieved_out": 0,
    "elapsed_sec": 5.6711000070208684e-05
  },
  {
    "n": 2684969071,
    "bits": 32,
    "attempts": 3,
    "sieved_out": 1,
    "elapsed_sec": 7.25869999769202e-05
  },
  {
    "n": 3489870187,
    "bits": 32,
    "attempts": 14,
    "sieved_out": 9,
    "elapsed_sec": 0.00013638500001889042
  },
  {
    "n": 2511292631,
    "bits": 32,
    "attempts": 1,
    "sieved_out": 0,
    "elapsed_sec": 5.108899995320826e-05
  },
  {
    "n": 2497839737,
    "bits": 32,
    "attempts": 8,
    "sieved_out": 7,
    "elapsed_sec": 5.8603000070434064e-05
  },
  {
    "n": 2172330977,
    "bits": 32,
    "attempts": 28,
    "sieved_out": 24,
    "elapsed_sec": 0.00013823800009049592
  },
  {
    "n": 2350185679,
    "bits": 32,
    "attempts": 26,
    "sieved_out": 24,
    "elapsed_sec": 8.723399992049963e-05
  },
  {
    "n": 18401742420732767113,
    "bits": 64,
    "attempts": 1,
    "sieved_out": 0,

```

```

    "elapsed_sec": 0.00012900199999421602
  },
  {
    "n": 15166909338898053059,
    "bits": 64,
    "attempts": 51,
    "sieved_out": 45,
    "elapsed_sec": 0.0003239269999539829
  },
  {
    "n": 12260737436038211209,
    "bits": 64,
    "attempts": 24,
    "sieved_out": 22,
    "elapsed_sec": 0.00014948699993055925
  },
  {
    "n": 16017586886302944473,
    "bits": 64,
    "attempts": 27,
    "sieved_out": 20,
    "elapsed_sec": 0.00028915300003973243
  },
  {
    "n": 11975328009848833073,
    "bits": 64,
    "attempts": 26,
    "sieved_out": 22,
    "elapsed_sec": 0.00022095099996022327
  },
  {
    "n": 13464028737317474003,
    "bits": 64,
    "attempts": 15,
    "sieved_out": 13,
    "elapsed_sec": 0.00015078000001267355
  },
  {
    "n": 11926652683290036799,
    "bits": 64,
    "attempts": 34,
    "sieved_out": 31,
    "elapsed_sec": 0.00019998599998416466
  },
  {
    "n": 14195471983499090161,
    "bits": 64,
    "attempts": 36,
    "sieved_out": 31,
    "elapsed_sec": 0.0002463100000795748
  },
  {
    "n": 67334772459302326248064735529,
    "bits": 96,
    "attempts": 111,
    "sieved_out": 98,
    "elapsed_sec": 0.0008047579999583832
  },
  {
    "n": 59972691807943756299763049609,
    "bits": 96,
    "attempts": 19,
    "sieved_out": 14,
    "elapsed_sec": 0.0003383310000799611
  },
  {
    "n": 39870876225218717285886667867,
    "bits": 96,
    "attempts": 13,
    "sieved_out": 10,
    "elapsed_sec": 0.00026800099999491067
  },
  {
    "n": 53188421132122500275852162669,

```

```

    "bits": 96,
    "attempts": 33,
    "sieved_out": 27,
    "elapsed_sec": 0.0004146289999198416
  },
  {
    "n": 45953160609077804534454216143,
    "bits": 96,
    "attempts": 18,
    "sieved_out": 14,
    "elapsed_sec": 0.0002932729998974537
  },
  {
    "n": 56551956778084646804055001517,
    "bits": 96,
    "attempts": 54,
    "sieved_out": 44,
    "elapsed_sec": 0.0005955479999784075
  },
  {
    "n": 66188843531002670023449553417,
    "bits": 96,
    "attempts": 77,
    "sieved_out": 69,
    "elapsed_sec": 0.00047302999996645667
  },
  {
    "n": 64870971140683346557985064293,
    "bits": 96,
    "attempts": 33,
    "sieved_out": 24,
    "elapsed_sec": 0.0005261610000388828
  },
  {
    "n": 255262491496932926240396306741840110429,
    "bits": 128,
    "attempts": 19,
    "sieved_out": 17,
    "elapsed_sec": 0.0003180999999585765
  },
  {
    "n": 179191618978837224285131028657492073939,
    "bits": 128,
    "attempts": 62,
    "sieved_out": 53,
    "elapsed_sec": 0.0008122970000385976
  },
  {
    "n": 284910772379906274498920410023681318383,
    "bits": 128,
    "attempts": 1,
    "sieved_out": 0,
    "elapsed_sec": 0.00022410199994737923
  },
  {
    "n": 303273034839925258263876384307280345587,
    "bits": 128,
    "attempts": 18,
    "sieved_out": 14,
    "elapsed_sec": 0.0004217679999101165
  },
  {
    "n": 303959982570498639628078674070035246173,
    "bits": 128,
    "attempts": 28,
    "sieved_out": 24,
    "elapsed_sec": 0.0003996720000714049
  },
  {
    "n": 281299237677265781840181265071800420149,
    "bits": 128,
    "attempts": 8,
    "sieved_out": 7,
    "elapsed_sec": 0.00022157900002639508
  }

```

```

    },
    {
      "n": 228253222684261744924825570484486262997,
      "bits": 128,
      "attempts": 29,
      "sieved_out": 25,
      "elapsed_sec": 0.0004462389999844163
    },
    {
      "n": 268709383189357700134964952562313026619,
      "bits": 128,
      "attempts": 32,
      "sieved_out": 29,
      "elapsed_sec": 0.00034304400003293267
    }
  ]
},
"naive_vs_practical": {
  "8": {
    "naive_succeeded": false,
    "seeded_succeeded": true
  },
  "12": {
    "naive_succeeded": true,
    "seeded_succeeded": true
  },
  "16": {
    "naive_succeeded": true,
    "seeded_succeeded": true
  },
  "20": {
    "naive_succeeded": false,
    "seeded_succeeded": true
  }
}
}

```

## Вывод

В ходе выполнения лабораторной работы были реализованы и экспериментально исследованы алгоритм комбинирования псевдослучайных последовательностей методом Макларена–Марсальи (вариант 1:  $N=10$ ,  $K=5$ ) и вероятностный тест простоты Соловея–Штрассена, интегрированный в полную схему построения большого простого числа с предварительным просеиванием пробным делением. Экспериментально подтверждено достаточное условие максимального периода линейного конгруэнтного генератора (§4.1): при его выполнении период под-генераторов достигает теоретического максимума ( $T=10$  и  $T=5$  соответственно) и подтверждается критерием хи-квадрат Пирсона (гипотеза о равномерности распределения не отвергается), а при нарушении условия период заметно сокращается. Вместе с тем практическая часть работы показала, что достижение максимального периода не гарантирует пригодности генератора для криптографических задач: абсолютная величина периода генератора Этапа 1 оказалась на порядки меньше числа различимых  $p$ -битных кандидатов уже при  $p \geq 16\text{--}20$ , вследствие чего в Этапе 2 генератор Этапа 1 корректно использован как источник сида, а корректность самого теста Соловея–Штрассена дополнительно подтверждена на классическом контрпримере теста Ферма — псевдопростом числе  $341 = 11 \cdot 31$ . Таким образом, цель работы — освоение программной реализации и практического применения алгоритмов генерации ПСП, методов их статистической оценки, а также вероятностных и детерминированных тестов простоты чисел с построением доказуемо/вероятно простых чисел — достигнута в полном объеме, а само исследование наглядно выявило границы применимости теоретических критериев качества генераторов псевдослучайных чисел.